

Expected Paragraphs

What did you do?

In this phase of the project, I focused on turning RetailSync from a foundation-level full-stack application into a more realistic workflow-based system. Earlier work was mostly about setting up authentication, tenant/company structure, permissions, and the initial database and module design. In this report period, I worked more on how the different parts of the system connect together in actual use cases.

The biggest progress was in the accounting workflow. I built and improved a statement-processing flow where a user can upload a bank statement PDF, store it in secure Google Cloud Storage, save the statement record in MongoDB, trigger background processing, extract OCR and structured data, detect checks, and review the results inside the app. I also improved the statement detail page so it now acts more like a review workspace, with a PDF viewer, artifact viewing, progress tracking, and better retry/error handling.

I also worked on QuickBooks integration, especially around CRUD completeness, authorization reliability, and how statement-driven accounting data relates to QuickBooks posting. In addition to product features, I spent time on deployment-related and infrastructure-related issues such as environment configuration, Google Cloud Storage bucket CORS, task dispatch behavior, and debugging background workflow failures. Overall, this phase was about making RetailSync behave more like a real connected product instead of a collection of separate modules.

How did you do it?

I built the project using a full-stack TypeScript setup. The frontend uses React, Redux Toolkit, and Material UI, while the backend uses Express, MongoDB, Mongoose, and Zod. I kept the project organized as a monorepo so shared schemas and validation rules could be reused between frontend and backend.

My workflow was usually to first define the shape of the data and validation rules, then implement the backend route or service, and finally connect it to the frontend. For workflow-heavy features like statement upload and extraction, I also had to think about storage paths, background jobs, retries, status tracking, and how the UI would represent long-running activity.

I used Google Cloud Storage for secure file upload and artifact storage, MongoDB for application state and workflow records, and background task logic for asynchronous processing. I also used tools like pnpm workspaces, Vitest, Docker/local environment setup, and cloud configuration debugging to keep development manageable. This phase required a lot of iterative debugging because many issues came from the interaction between frontend, backend, storage, and third-party integrations rather than from a single file.

What problems did you encounter?

One of the biggest challenges was that the project now has multi-step workflows, which means bugs can happen across several layers at once. For example, a statement PDF might upload successfully, but the later processing could still fail because of storage-read problems, queue dispatch issues, or invalid environment configuration. That made debugging more difficult because I had to trace the full path from browser upload to cloud storage, MongoDB persistence, task execution, artifact generation, and frontend display.

I also encountered problems with Google Cloud Storage bucket configuration, especially around CORS for browser uploads. Since the upload flow depends on signed URLs and direct browser-to-storage communication, the bucket settings had to match the local frontend origin correctly. I also had issues with background processing and QuickBooks authorization, including degraded OAuth states and environment-dependent failures.

Another challenge was UI trust and visibility. When a workflow takes time, the user needs to see what stage is happening or it feels like the system is frozen. Because of that, I had to improve the UX around loading, processing activity, retries, and saved artifacts. This phase showed me that workflow software needs both backend correctness and clear frontend feedback to feel reliable.

What Did You Do

What features did you finish?

I finished or significantly improved the bank statement workflow, including secure PDF upload, MongoDB statement persistence, processing status handling, background extraction flow, artifact tracking, and statement review UI. I also improved the statement detail page with embedded artifact viewing, clearer progress indicators, and a better activity/debugging experience.

I also completed more of the QuickBooks integration by confirming supported transaction flows and adding missing CRUD coverage for the manual QuickBooks write path. In addition, I fixed infrastructure and workflow issues involving Google Cloud Storage uploads, bucket CORS behavior, environment configuration, and local task dispatch behavior.

Can the features be demoed?

Yes, the features can be demoed. A demo can now show login, tenant-aware access, a bank statement upload, storage of the PDF in Google Cloud Storage, creation of the statement record in MongoDB, progress through extraction and review, and viewing the results inside the app. A second demo path can show QuickBooks-connected operations and how the accounting workspace now behaves more like a real end-to-end system.

How many of the features are integrated in a way where they can be used together?

A large portion of the important features are now integrated. Authentication, company scoping, permissions, statement upload, cloud storage, MongoDB persistence, background processing, artifact generation, review UI, and QuickBooks-related operations now connect to each other in a usable workflow. Not every part of the system is fully complete yet, but it is no longer just separate modules; several core features now work together as one flow.

How Did You Do It

Did you use any project templates or tutorials?

I did not use a full project template, but I followed common patterns from React, Express, MongoDB, and TypeScript projects. I also used official documentation and examples for features like authentication, signed uploads, storage configuration, API design, and frontend file handling.

Did you seek help from the instructor or other colleagues?

I mainly implemented the features independently, but I did use outside input when I needed help thinking through architecture decisions, debugging, or workflow design. Discussions with others and reference material were useful for validating better approaches, especially when dealing with larger structural decisions or integration issues.

Did you use any AI/ML/LLMs?

Yes, I used AI tools during development. I used them mainly for debugging support, code analysis, implementation ideas, and understanding integration problems faster. However, I still tested and verified everything in the actual project before treating it as complete.

What Problems Did You Encounter

Do you have any concerns about your progress?

My main concern is managing complexity as the system grows. The project now includes authentication, permissions, data import, storage, OCR/extraction, background jobs, review UX, and QuickBooks integration. Because of that, I need to keep improving test coverage, observability, and overall project structure so the system remains maintainable and does not become fragile.

Do you need any help currently?

The most useful help right now would be feedback on workflow design, architecture decisions, and best practices for scaling a larger full-stack application with integrations. Guidance on which workflows should be prioritized next would also be helpful, since the project is now at the stage where integration quality matters as much as individual features.